⚠️ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

This folder is the training site's **own** IEC 62304 lifecycle document set — the site turned on itself. The Learn page's deliverables list answers "what documents does IEC 62304 require at a given safety class?" from `data/applicability.json`. This folder is the worked answer to the next question: what does it actually look like to produce them, for a real (if small) piece of software?

What this is, and is not

This repository is a training website, not a medical device — under any regulatory definition, in any jurisdiction. It is not Software as a Medical Device (SaMD), it is not Software in a Medical Device (SiMD), and nothing here is submitted to a regulator. Nothing in this folder should be read as a claim that `stjohnlynch.com` 's training site *is* medical device software. It is not.

What follows is a **demonstration document set**: this project's own software development, documented as if it were being run under IEC 62304, so that a reader learning the standard from the Learn page can also see what the paperwork the standard asks for actually contains, rather than only reading about it in the abstract. Where the project actually has the evidence a document calls for — a test suite, a design record, a problem log — that evidence is cited directly rather than restated. Where it does not, the document says so plainly rather than inventing content to fill the gap; a document set with no visible gaps would itself be a worse teaching example, since real projects have gaps and the standard's Clause 9 exists precisely to track them rather than pretend they don't happen.

Why Class C

You were asked which class to target and picked **Class C** — every one of the 13 process areas applies in full, which makes this the largest and most complete version of the set. §4.3 classifies by asking whether a software item could contribute to a hazardous situation and, if so, how severe the resulting harm could be; this site cannot literally cause harm to anyone, so a real classification would land it outside IEC 62304's scope entirely. Class C here is a **deliberately chosen teaching target**, not a finding — see 01 — General Requirements for the walkthrough of why, done the way §4.3 actually asks you to do it.

The document set

One file per process area, in clause order, each following the same shape: what the standard requires (sourced from `applicability.json`, not paraphrased), how this project currently satisfies it, and plainly, what it does not yet.

#	DOCUMENT	CLAUSE	STATUS
01	General Requirements & Classification	Clause 4	Documented
02	Software Development Plan	Clause 5.1	Documented
03	Software Requirements Specification	Clause 5.2	Documented
04	Software Architecture	Clause 5.3	Documented
05	Software Detailed Design	Clause 5.4	**Partial — see gaps**
06	Unit Implementation & Verification	Clause 5.5	**Partial — see gaps**
07	Software Integration & Testing	Clause 5.6	**Partial — see gaps**

#	DOCUMENT	CLAUSE	STATUS
08	Software System Testing	Clause 5.7	Documented — strong evidence
09	Software Release	Clause 5.8	**Partial — see gaps**
10	Software Maintenance Plan	Clause 6	Documented
11	Software Risk Management File	Clause 7	Documented — reflexive
12	Software Configuration Management Plan	Clause 8	Documented
13	Software Problem Resolution Process	Clause 9	Documented — strong evidence

This set is reflexive by design — see "What this is, and is not" above. For a worked example that instead documents a fictional *medical device*, with the hazards, essential performance and hardware/software interface this set structurally cannot provide, see the device example register — that is the set the Learn page's Preview/Download buttons link to; this reflexive set is still real and still worth reading, but is reached from this page rather than from a clause card.

Four documents are marked **Partial**: this project does not do isolated unit testing of its JavaScript (Clause 5.5), does not separate integration testing from system testing because it has no formally partitioned software items to integrate (Clause 5.6), only has detailed design written down for one of six scripts (Clause 5.4), and has no formal release-versioning scheme — no version tags, no changelog (Clause 5.8). Each of those documents says so in its own **Gaps** section rather than papering over it, which is the more useful thing for a document in this set to demonstrate: the standard's own answer to "we found a gap" is Clause 9's problem resolution process, not silence.

Reading these documents online, or on paper

Each document has two forms, generated from the same markdown source so they never disagree: a normal web page (**Preview**) and a PDF (**Download**), both linked from the table above. The Learn page's own Preview/Download buttons link to the device example register instead of this set — see that register for why. Neither form requires a GitHub account or any familiarity with markdown — this course is written for people who work in Word, Excel, PowerPoint and PDF, and the previous version of this download offered the raw `.md` source instead, which was source code to that audience, not a document. Within a document's own text, mentions of source files (`tests/test_site.py` , `data/*.json` , this project's design notes) are plain text, not links — a link to a raw, unrendered file is a dead end for a reader without a code editor, so the file is named for context without being offered as something to click through to.

How to read "applies to Class C" in these documents

A sub-clause "applies to Class C" if its `classes` array in `applicability.json` contains `"C"` . In this data set that is true of every sub-clause that has any requirement at all — `["C"]` , `["B","C"]` , and `["A","B","C"]` are all included; only `7.1.5` and `7.3.2` , which Amendment 1 deleted outright, are excluded. So each table below is, in effect, "every live requirement of this clause." See the site's own Safety Class Applicability section for why the mapping is recorded at this level of detail in the first place.

Cross-references used throughout

- **`applicability.json`** — the regulatory mapping; `ref` , `title` , `classes` and `output` fields are quoted verbatim in each document's requirements table.
- **DESIGN.md** — page-by-page design rationale, JavaScript architecture, and the safety-classification defect narrative referenced in 11 — Risk Management File.
- **README.md** — features, testing, and the anomaly log.
- **`tests/test_site.py`** — the system test suite (606 checks across 12 groups) that is this project's primary verification evidence.

- **`tests/anomaly_log.csv`** — the standing problem log described in 13 — Problem Resolution.
- **`git log`** — the configuration history; see 12 — Configuration Management Plan.